

实验五：鸿蒙多端应用开发

实验项目基本信息

- 实验编号：d20301035105、d20301009205
- 学时分配：4学时
- 实验类型：验证型
- 每组人数：6人
- 对应课程目标：课程目标3

核心步骤列表

实验概览

- 需求分析阶段：定义天气应用功能需求与多端适配策略，明确传感器数据采集需求
- AI辅助编码阶段：使用TRAE辅助生成ArkUI页面布局与传感器调用代码
- 测试验证阶段：在不同设备模拟器上验证UI适配效果与传感器数据准确性
- 部署运维阶段：编写多端适配与硬件集成技术分析报告

开发任务清单

任务阶段	主要内容	关键技术点
天气应用	ArkUI开发	@State、数据绑定
多端适配	响应式布局	断点系统、媒体查询
传感器	光线/陀螺仪	传感器API调用
分布式	数据同步	分布式能力演示

成功标准

- ☐ 天气应用功能完整
- ☐ 手机/平板界面自适应
- ☐ 传感器数据正常采集
- ☐ 多端适配效果良好
- ☐ 技术分析报告完成

实验任务与案例

核心任务

使用DevEco Studio开发天气应用，实现手机/平板界面自适应布局；通过模拟器演示分布式数据同步原理。扩展案例：调用设备传感器（光线传感器、陀螺仪）实现简单的物联网数据采集与展示场景。

实战案例

开发"智慧天气"应用，展示当前天气信息，在手机和平板上自适应显示，集成传感器展示环境数据，体验鸿蒙多端统一开发能力。

第一课时：天气应用开发与多端适配（2学时）

5.1 需求分析阶段

步骤1：定义功能需求

1. 天气展示

- 城市名称
- 温度
- 天气状况
- 空气质量

2. 多端适配

- 手机竖屏布局
- 平板横屏布局
- 自适应断点

3. 传感器需求

- 光线传感器：自动亮度
- 陀螺仪：动态效果

5.2 创建鸿蒙项目

步骤1：新建项目

- 打开DevEco Studio
- 新建Empty Ability项目
- 选择ArkTS语言
- 命名为WeatherApp

5.3 AI辅助编码

步骤1：生成天气页面代码

1. 使用TRAE描述需求：ArkTS天气应用页面，声明式UI，城市温度展示
2. 生成代码片段
3. 手动完善

步骤2：实现天气首页

```
// ets/pages/index/Index.ets
@Entry
@Component
struct Index {
  @State cityName: string = '合肥';
  @State temperature: number = 22;
  @State weather: string = '晴';
  @State aqi: number = 45;

  build() {
    Column() {
      // 城市名称
      Text(this.cityName)
        .fontSize(24)
        .fontWeight(Fontweight.Bold)
        .margin({ top: 50 })

      // 天气图标
      Text(this.weather === '晴' ? '☀️' : '☁️')
        .fontSize(80)
        .margin({ top: 20 })

      // 温度
      Text(`${this.temperature}°C`)
        .fontSize(60)
        .fontWeight(Fontweight.Lighter)
        .margin({ top: 20 })

      // 天气状况
      Text(this.weather)
        .fontSize(20)
        .margin({ top: 10 })

      // 空气质量
      Row() {
        Text('空气质量: ')
        Text(`${this.aqi} - 优`)
          .fontColor(this.aqi <= 50 ? '#00ff00' : '#ff9900')
      }
      .margin({ top: 30 })

      // 传感器数据展示
```

```

    Row() {
        Text('光线传感器: ')
        Text(this.lightValue.toString())
            .id('lightvalue')
    }
    .margin({ top: 20 })
}
.width('100%')
.height('100%')
.backgroundColor('#f5f5f5')
}

@State lightValue: number = 0;
}

```

5.4 多端适配布局

步骤1：响应式布局实现

```

// 断点系统适配
@State currentBreakpoint: string = 'md';

aboutToAppear() {
    // 获取设备类型
    let deviceInfo = umpInfo.getDeviceInfo();
    let windowWidth = window.getWindow().getWindowProperties().windowWidth;

    if (windowWidth < 600) {
        this.currentBreakpoint = 'sm'; // 手机
    } else if (windowWidth < 840) {
        this.currentBreakpoint = 'md'; // 平板竖屏
    } else {
        this.currentBreakpoint = 'lg'; // 平板横屏
    }
}

build() {
    if (this.currentBreakpoint === 'sm') {
        // 手机布局
        Column() {
            this.phoneLayout()
        }
    } else {
        // 平板布局
        Row() {
            this.tabletLayout()
        }
    }
}

@Builder phoneLayout() {
    Column() {

```

```
        Text(this.cityName).fontSize(24)
        Text(`${this.temperature}°C`).fontSize(60)
    }
}

@Builder tabletLayout() {
    Column() {
        Text(this.cityName).fontSize(32)
        Text(`${this.temperature}°C`).fontSize(80)
    }
    .width('40%')
}
```

第二课时：传感器集成与分布式能力（2学时）

5.5 光线传感器开发

步骤1：导入传感器模块

```
import sensor from '@ohos.sensor';
```

步骤2：实现光线传感器

```
@State lightIntensity: number = 0;

aboutToAppear() {
    // 注册光线传感器监听
    sensor.on(sensor.SensorType.SENSOR_TYPE_ID_LIGHT, (data) => {
        this.lightIntensity = data.light;
    });
}

aboutToDisappear() {
    // 取消监听
    sensor.off(sensor.SensorType.SENSOR_TYPE_ID_LIGHT);
}

// UI展示
Text(`环境光线: ${this.lightIntensity} lux`)
    .fontSize(16)
    .margin({ top: 10 })
```

5.6 陀螺仪传感器开发

步骤1: 实现陀螺仪数据

```
@State rotateX: number = 0;
@State rotateY: number = 0;
@State rotateZ: number = 0;

aboutToAppear() {
  sensor.on(sensor.SensorType.SENSOR_TYPE_ID_GYROSCOPE, (data) => {
    this.rotateX = data.x;
    this.rotateY = data.y;
    this.rotateZ = data.z;
  });
}

// UI展示
Row() {
  Column() { Text(`X轴: ${this.rotateX.toFixed(2)}°`) }
  Column() { Text(`Y轴: ${this.rotateY.toFixed(2)}°`) }
  Column() { Text(`Z轴: ${this.rotateZ.toFixed(2)}°`) }
}
.spacing(20)
```

5.7 分布式数据同步演示

步骤1: 分布式数据管理

```
import distributedData from '@ohos.data.distributedData';

// 创建分布式数据库
const KVManager = distributedData.createKVManager({
  bundleName: 'com.example.weather',
  kvStoreType: distributedData.KVStoreType.SINGLE_VERSION
});

// 存储数据
async function saveData(key: string, value: string) {
  const kvStore = await KVManager.getKVStore('weatherStore');
  await kvStore.put(key, value);
}

// 读取数据
async function getData(key: string) {
  const kvStore = await KVManager.getKVStore('weatherStore');
  return await kvStore.get(key);
}
```

5.8 测试验证

步骤1：多设备模拟器测试

- 1. 启动手机模拟器
- 2. 启动平板模拟器
- 3. 测试界面适配效果
- 4. 观察传感器数据

步骤2：传感器数据验证

- 1. 模拟光线变化
- 2. 模拟设备旋转
- 3. 验证数据准确性

5.9 技术分析报告

步骤1：编写分析报告

```
# 多端适配与硬件集成技术分析报告

## 1. 多端适配分析
- 手机布局特点
- 平板布局特点
- 断点系统实现

## 2. 传感器集成分析
- 光线传感器应用场景
- 陀螺仪应用场景
- 数据采集准确性

## 3. 分布式能力分析
- 分布式数据管理原理
- 跨设备同步实现

## 4. 技术选型建议
```

总结与思考

实验总结

- 掌握ArkUI声明式开发
- 学会多端响应式布局
- 了解传感器数据采集
- 理解分布式能力原理

课后思考

1. 鸿蒙多端开发相比其他框架的优势
2. 传感器在物联网中的应用场景
3. 分布式技术对未来移动开发的影响